COMP3308/3608 Week 3: Informed and Local Search Algorithms

Kenny Yu

2026 Semester 1

Contents

1 A* Algorithm

1.1 A* Search

In search algorithms covered last week, two important strategies appear repeatedly:

- **Uniform-Cost Search (UCS)** expands the node with the smallest path cost accumulated so far.
- **Greedy Search (GS)** expands the node that appears closest to a goal according to a heuristic estimate.

These two ideas capture different priorities:

- UCS is careful about the **past**: it values low cost already incurred.
- Greedy search is optimistic about the **future**: it values low estimated remaining cost.

A* combines both ideas into a single evaluation rule.

Definition 1.1.1 (A*)**.** For each node n, define:

- $g(n)$: thr path cost from the start node to n
- $h(n)$ the heuristic estimate of the cost from n to a goal
- $f(n) = g(n) + h(n)$: the estimated total cost of a solution path passing through n

A* always selects for expansion the fringe node with the smallest f-value.

The quantity $f(n)$ is not the true solution cost through n, but an estimate of it since $g(n)$ is known exactly for the current path but $h(n)$ is an estimate of what remains.

Hence, A* behaves like a principled compromise between caution and optimism.

Generic Algorithmic Idea

1. Put the start node in the fringe.
2. Repeatedly remove the node with smallest f-value.
3. If that node is a goal, terminate.

4. Otherwise expand it, compute g, h, and f for its successors, and insert them into the fringe.
5. Continue until a goal is selected for expansion or the fringe becomes empty.

Example 1.1.1. Recall the Romania route-finding problem from **Arad** to **Bucharest**.

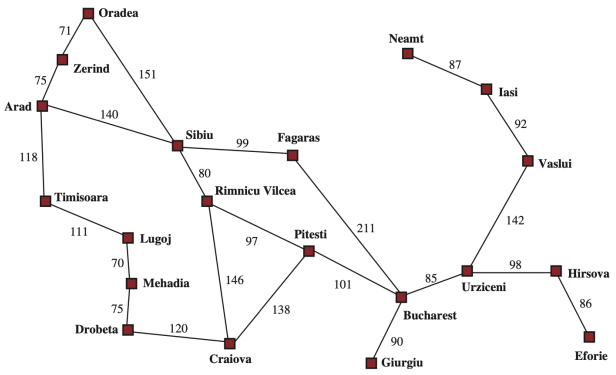

Figure 1: A simplified road map of part of Romania, with road distances in miles.

- A* uses the current road cost from Arad to a city as $g(n)$.
- It uses the straight-line distance to Bucharest as $h(n)$.
- It then expands the city with smallest $f(n) = g(n) + h(n)$

Example 1.1.2. Given a graph with goal nodes C, I, E, K. The edge labels represent step costs, and the node labels in brackets represent heuristic values. Assume the ties are broken by expanding the last-added node first.

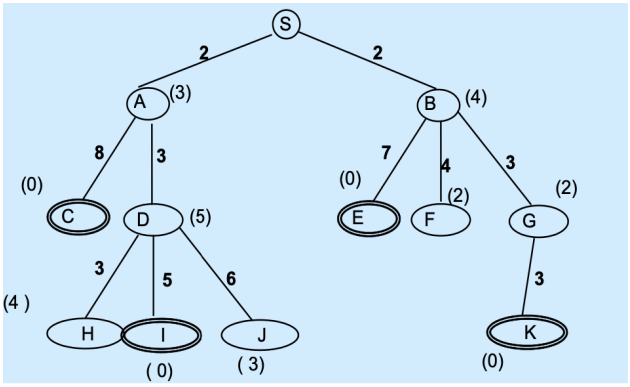

Figure 2: Example graph for running A*.

Running A* yields:

- expanded nodes: S, A, B, G, K
- solution path: $S \to B \to G \to K$
- solution cost: 8

1.2 A*, UCS, and BFS

A* is best understood as a general scheme. Several familiar search algorithms arise as special cases of A* under particular choices of the evaluation function.

UCS as a special case of A*

UCS expands nodes by smallest $g(n)$, A* expands nodes by smallest

$$f(n) = g(n) + h(n).$$

Therefore, if $h(n) = 0$ for all n, then $f(n) = g(n)$ and A* becomes UCS.

BFS as a special case of UCS

If every step cost is 1, then path cost equals depth:

$$g(n) = \text{depth}(n).$$

Hence, UCS behaves like BFS is that situation.

Relationship Summary

A useful nesting is:

- A* is the most general form.
- UCS is A* with $h(n) = 0$.
- BFS is UCS when $g(n) = \text{depth(n)}$, equivalently unit step costs.

So the hierarchy is

$$\text{BFS} \subseteq \text{UCS} \subseteq \text{A}^*.$$

1.3 Admissible Heuristics

Definition 1.3.1 (Admissibility)**.** A heuristic $h(n)$ is **admissible** if, for every node n, it never overestimates the true cost to reach a goal. Let $h^*(n)$ denote the true minimum cost from n to a goal. Then admissibility means:

$$h(n) \leq h^*(n) \quad \text{for all } n.$$

An admissible heuristic is **optimistic**. It always assumes the remaining cost is no more than it really is. This optimism is exactly what allows A* to make safe decisions while still being guided.

Example 1.3.1 (Straight-line distance)**.** In route-finding, the straight-line distance from a city to the destination is admissible if travel is constrained to roads, because the road distance cannot be shorter than the straight-line distance. Thus, straight-line distance is a lower bound on true travel cost.

Why admissibility matters?

Theorem 1.3.1. If h is admissible, then A* is complete and optimal. That is

- **complete**: it will find a solution if one exists (under the standard finite-cost assumptions)
- **optimal**: the first goal it selects for expansion is guaranteed to be a least-cost solution

Example 1.3.2. Consider the graph in Figure 2. The heuristic is checked by comparing each node's heuristic value to the true shortest remaining path cost to a goal. Let the heuristic value of the starting node S be 5.

- $h(S) = 5 \leq 8$ (shortest path from S to a goal, i.e. to goal K)
- $h(B) = 4 \leq 6$
- $h(G) = 2 \leq 3$
- $h(A) = 3 \leq 8$
- $h(D) = 5 \leq 5$

Hence the heuristic h is admissible.

This demonstrates the exact procedure:

1. Compute or reason about the true remaining optimal cost $h^*(n)$.
2. Check that $h(n) \leq h^*(n)$ for each relevant node.

Proof (A* is optimal with an admissible heuristic)

Suppose there are two goals: G an optimal goal, G_2 a suboptimal goal. We want to show that A* cannot select G_2 for expansion before some node on the optimal path to G.

Since goals have $h = 0$:

$$f(G_2) = g(G_2) \quad \text{and} \quad f(G) = g(G).$$

Because G_2 is suboptimal, $g(G_2) > g(G)$, hence

$$f(G_2) > f(G).$$

Let n be an unexpanded fringe node on the optimal path to G. By admissibility:

$$h(n) \leq h^*(n)$$

so

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n).$$

But $g(n)+h^*(n)$ is exactly the cost of the optimal solution through n, which equals $g(G)$. Therefore:

$$f(n) \leq g(G) = f(G).$$

Since $f(G) < f(G_2)$, we obtain

$$f(n) < f(G_2).$$

Thus, A* will expand n before G_2, so G_2 cannot be the first goal selected.

The proof shows that admissibility prevents A* from being misled into committing too early to a suboptimal goal. There is always some node on the optimal route whose f-value is at least as good as the optimal goal and strictly better than any suboptimal goal.

1.3.1 Admissible Heuristics for the 8-Puzzle

The 8-puzzle consists of eight numbered tiles and one blank arranged on a 3×3 board. A move slides a tile into the blank space. An example instance is shown in Figure 3

The objective is to reach the goal arrangement with minimum number of moves.

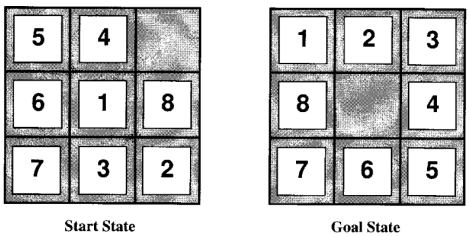

Figure 3: An example of 8-puzzle.

Define:

$$h_1(n) = \text{number of misplaced tiles.}$$

Every misplaced tile must be moved at least once before the puzzle can be solved. Therefore the number of misplaced tiles is a lower bound on the number of moves still required, i.e. h_1 never overestimates true remaining cost. Hence, h_1 is **admissible**.

Now we define:

$$h_2(n) = \sum_i \text{ManhattanDistance}(\text{tile}_i, \text{ goal position of tile}_i)$$

The Manhattan distance counts horizontal and vertical displacement only. In the example illustrated in Figure 3,

$$h_2(\text{start}) = 2 + 3 + 3 + 2 + 4 + 2 + 0 + 2 = 18.$$

Each tile must be moved at least enough times to cover its horizontal and vertical displacement to its goal position. Summing these lower bounds across tiles gives a lower bound on the total number of moves. The true cost may be larger because the blank's position imposes constraints, but it cannot be smaller than this sum. So h_2 is **admissible**.

Usually h_2 is more informative than h_1, because it measures not merely whether a tile is misplaced, but how far away it is. That leads naturally to the concept of **dominance**.

1.4 Dominant Heuristics

Definition 1.4.1 (Dominance of heuristics)**.** Let h_1 and h_2 be admissible heuristics for the same problem. We say that h_2 **dominates** h_2 if:

$$h_2(n) \geq h_1(n) \quad \text{for all } n.$$

Since both are admissible, neither overestimates. Therefore the dominating one gives estimates that are uniformly closer to the true cost.

Theorem 1.4.1. If h_2 dominates h_1, then A* using h_2 expands fewer nodes than A* using h_1. Equivalently, the more informed admissible heuristic is better for search efficiency.

A node must often be expanded when its f-value is below the optimal solution cost threshold. A larger admissible heuristic raises $f(n) = g(n) + h(n)$, which can keep more irrelevant nodes from looking promising. Thus, a dominating heuristic prunes more of the search space without sacrificing optimality.

Example 1.4.1 (8-Puzzle)**.** For the 8-puzzle, Manhattan distance typically dominates misplaced tiles:

$$h_2(n) \geq h_1(n)$$

because every misplaced tile contributes at least 1 to Manhattan distance.

Typical 8-puzzle search cost at solution depth 14:

- IDS: 3,473,941 nodes
- A* with h_1: 539 nodes
- A* with h_2: 113 nodes

This concretely shows how a stronger heuristic dramatically reduces search effort.

1.5 Combining Heuristics

Suppose h_1 and h_2 are admissible. Define:

$$h_3(n) = \min\{h_1(n), h_2(n)\}, \quad h_4(n) = \max\{h_1(n), h_2(n)\}.$$

Both h_3 and h_4 are **admissible**, because the minimum of two lower bounds is still a lower bound; and the maximum of two lower bounds is also still a lower bound, since each individual value is already no greater than the true cost.

Since $h_4(n) \geq h_3(n)$ for all n, h_4 **dominates** h_3. Therefore h_4 is a better heuristic for search.

General composite heuristic

If we have several admissible heuristics $h_1, \ldots, h_m$, and no single one dominates all others, a standard idea is to define:

$$h(n) = \max\{h_1(n), h_2(n), \ldots, h_m(n)\}.$$

This composite heuristic is admissible and is at least as informed as each component.

1.6 Constructing Admissible Heuristics from Relaxed Problems

A powerful way to invent admissible heuristics is:

1. Remove some constraints from the original problem.
2. Solve the relaxed problem exactly.
3. Use that exact solution cost as the heuristic for the original problem.

The relaxed problem is easier because more actions are permitted. Therefore its optimal solution cost can only be less than ore equal to the optimal cost of the original problem. Hence it is a lower bound, and therefore an admissible heuristic.

Theorem 1.6.1. The optimal solution cost to a relaxed problem is an admissible heuristic for the original problem.

1.6.1 8-Puzzle examples

Relaxation 1: *Allow any tile to move anywhere.*
Then the exact cost becomes the number of misplaced tiles, giving heuristic h_1.

Relaxation 2: *Allow any tile to move to any adjacent square.*
Then the exact cost becomes Manhattan distance, giving heuristic h_2.

1.6.2 Automatic construction of relaxations

If the problem is formally described, one can sometimes generate relaxed problems automatically by removing conditions from action preconditions.

For the 8-puzzle example, the usual move rule might be:

A tile may move from square A to square B if:

1. *A is adjacent to B, and*
2. *B is blank.*

Removing conditions yields relaxed variants such as:

- adjacency required, blank not required
- blank required, adjacency not required
- neither required

ABSOLVER (1993) is a program that can generate heuristics automatically using the "relaxed problem" method and other methods.

1.6.3 Learning heuristics from experience

Another idea is to **learn** a heuristic from solved examples.

For the 8-puzzle, we could build a dataset of states with known true solution costs and features such as: number of misplaced tiles, and number of adjacent tiles that should not be adjacent. Then a learning algorithm predicts h from these features. However, the learned heuristic is not guaranteed to be admissible or consistent.

1.7 Consistent (Monotonic) Heuristics

Definition 1.7.1 (Consistent Heuristics)**.** A heuristic h is **consistent** if for every parent-child pair (n_i, n_j):

$$h(n_i) \leq c(n_i, n_j) + h(n_j) \tag{1}$$

where $c(n_i, n_j)$ is the step cost from parent to child.

Consistency means that the estimated remaining cost cannot drop too sharply in a single step. Rearranging the inequation 1 gives

$$h(n_j) \geq h(n_i) - c(n_i, n_j).$$

So when moving from parent to child, the heuristic estimate can decrease, but by no more than the step cost just paid.

Geometric intuition

This is analogous to metric distance:

distance from parent to goal $\leq$ cost from parent to child + distance from child to goal.

Thus consistency says the heuristic behaves like a well-behaved distance estimate over the search graph.

Consistency and non-decreasing f-values

Theorem 1.7.1. If h is consistent, then along any path:

$$f(n_j) \geq f(n_i)$$

for every parent-child pair (n_i, n_j). That is, f-values never decrease along a path.

Proof sketch. Since $g(n_j) = g(n_i) + c(n_i, n_j)$, we have

$$f(n_j) = g(n_j) + h(n_j) = g(n_i) + c(n_i, n_j) + h(n_j).$$

Since h is consistent, using inequality 1:

$$f(n_i) = g(n_i) + h(n_i) \leq g(n_i) + c(n_i, n_j) + h(n_j) = f(n_j).$$

Therefore

$$f(n_i) \leq f(n_j).$$

Converse. If f-values are non-decreasing along every path, then the heuristic is consistent. Hence the two conditions are equivalent.

$$h \text{ is consistent} \iff f \text{ is non-decreasing}$$

1.8 Relationship between Admissibility and Consistency

Consistency is a stronger requirement than admissibility.

Theorem 1.8.1. For any heuristic:

$$\text{consistency} \implies \text{admissibility}$$

but the converse is not necessarily true.

Admissibility is enough to guarantee that A* is optimal. Consistency gives more structure:

- f-values increase monotonically,
- search behaves more regularly,
- and A* enjoys stronger efficiency guarantees.

1.9 Completeness, Optimality, and Efficiency of A* with consistent Heuristics

Contour interpretation

If h is consistent, then f-values are non-decreasing. We can imagine the state space partitioned into f-**contours**:

- contour $f \leq 300$
- contour $f \leq 400$
- contour $f \leq 420$, etc.

A* expands nodes in increasing order of contour value.

Completeness intuition

As A* gradually expands contours of larger and larger f, eventually it must reach the contour containing an optimal goal G, for which

$$f(G) = g(G) + h(G) = g(G)$$

because $h(G) = 0$. Thus A* eventually finds a solution.

Optimality intuition

Suppose two goal nodes lie on different contours, with:

$$f(G_2) < f(G_3).$$

Since both are goals, $h(G_2) = h(G_3) = 0$, so:

$$g(G_2) < g(G_3).$$

Therefore the first goal reached by increasing f-contours is the optimal goal.

Optimal Efficiency

Theorem 1.9.1. If h is consistent, then A* is **optimally efficient** among all optimal search algorithms using the same heuristic. That means no other optimal algorithm using h is guaranteed to expand fewer nodes than A*.

This is a very strong result: with a consistent heuristic, A* is not only correct, but in precise sense cannot be systematically improved upon while retaining optimality.

1.10 Overall Properties of A*

Completeness

A* is complete, unless there are infinitely many nodes with $f \leq f(G)$, where G is the optimal goal. Under ordinary finite branching and positive step-cost assumptions, A* is complete.

Optimality

A* is optimal if the heuristic is admissible.

Time Complexity

In the worst case, time is exponential: $O(b^d)$, where b is branching factor and d is depth of solution.

1.10.1 Space Complextiy

Space is also exponential because A* stores all generated nodes in memory. In practice, **space is often the bigger problem**.

Practical consequences

Even though A* is theoretically excellent, it can become impractical on large problems. This motivates memory-bounded variants such as

- IDA* (Iterative Deepening A*)
- SMA* (Simplified Memory-Bounded A*)

Important engineering perspective:

- a dominant admissible heuristic may be more accurate, but also more expensive to compute
- sometimes a simpler heuristic that expands more nodes can still be faster overall
- if no good admissible/consistent heuristic exists, a non-admissible heuristic or a local search method may work better in practice

This is a classic trade-off between theory and practical efficiency.

2 Local Search Algorithms

2.1 Optimisation Problems

Up to this point, search was mainly **path finding**:

- start state S
- goal state G
- solution: a path from S to G
- optimal solution: least-cost path

In **optimisation problems**, the focus changes.

Definition 2.1.1 (Optimisation problem)**.** For an optimisation problem, each state has an evaluation score v, determined by an objective function. The goal is to find the state with the **highest** (global maximum) (or **lowest** (global minimum)) value depending on the application. The solution is the **state itself**, not the path leading to it.

Optimisation spaces may be extremely large. Enumerating states systematically may be too expensive. Therefore, algorithms that inspect only a tiny part of the state space are often preferred. This leads to **local search algorithms**.

Example 2.1.1 (n-Queens)**.** A classic example is the n-queens problem.

- initial state: some arrangement of n queens on the board
- states: all arrangements with n queens on the board
- goal: no queen attacks any other queen
- operators: move a queen, or move a queen in a way that reduces attacks

Here the sequence of moves does not matter. Only the final board matters.

Value Landscape

The objective function induces a **landscape** over the state space:

- peaks correspond to maxima

- valleys correspond to minima
- there may be local extrema

Two desirable properties are:

- **complete local search**: finds a goal state if one exists
- **optimal local search**: finds the best goal state, i.e. the global optimum

Most local search methods sacrifice one or both guarantees in exchange for scalability.

2.2 Hill-Climbing

Hill-climbing is an **iterative improvement** algorithm. It keeps only one current state in memory and repeatedly tries to replace it with a better neighbouring state. There are two common versions:

- **steepest ascent**: maximise v
- **steepest descent**: minimise v

2.2.1 Intuition

Hill-climbing is like climbing Everest in thick fog with amnesia:

- **thick fog**: the algorithm sees only immediate neighbours, not the whole landscape
- **amnesia**: it remembers only the current state, not the path or previous alternatives

2.2.2 Generic procedure (descent version)

1. Set current node n to the initial state s.
2. Generate successors of n; let n_{best} be the successor with best value.
3. If n_{best} is not better than n, stop and return n.
4. Otherwise set $n \leftarrow n_{\text{best}}$ and repeat.

The ascent version is identical except "better" means larger rather than smaller values.

Hill climbing always chooses the best immediate successor and never backtracks. This makes it memory-efficient but also vulnerable.

Example 2.2.1. Consider the problem shown in Figure 4. Hill-climbing expands: S, A, F, because at each step it selects the best child according to the objective function.

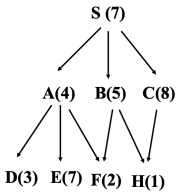

Figure 4: Example problem for hill-climb demonstration.

2.2.3 Strengths and Weaknesses of Hill-Climbing

Advantages

- Very little memory usage.
- Simple to implement.
- Often finds reasonable solutions in huge spaces where systematic search is impossible.

Weaknesses

- **Not complete**.
- **Not optimal**.
- Easily gets stuck in local optima.

A local optimum may be acceptable in practice, but there is no guarantee it is globally the best.

Dependence on initial state. The solution found depends heavily on the starting point. Starting near different parts of the landscape may lead to completely different final states.

2.3 Escaping Bad Local Optima in Hill-Climbing

2.3.1 Random restart

A simple remedy is **random restart**:

1. Run hill-climbing from one random initial state.
2. Record the local optimum reached.

3. Restart from another random initial state.
4. Return the best result found across runs.

This is effective when no fixed start state is required.

2.3.2 Plateaus

A **plateau** is a flat region where neighbouring states have the same or nearly the same value.

Problems caused by plateaus:

- the algorithm may stop immediately if it requires strict improvement
- or it may wander endlessly if sideways moves are allowed without control.

It is important to track repeated same-value situations and preventing endless revisiting.

2.3.3 Ridges

A **ridge** is a situation where the landscape slopes upward overall, but every individual legal move goes downhill. Thus, the algorithm cannot climb further even though better states exist nearby in a broader sense.

Possible remedies include:

- macro-moves that combine several primitive moves
- limited look-ahead search

These allow the algorithm to cross the awkward geometry of the landscape.

2.4 Beam Search

Beam search is a local-search-style strategy that keeps track of k candidate states rather than just one. In essence: keep only k best states at each stage.

2.4.1 Two Variants

Version 1: One Given Start State

- Start from one initial state.
- Generate all successors of the retained states at the current level.
- If a goal is found, stop.
- Otherwise keep only the best k successors.

Version 2: k Random Start States

- Start from k random states.
- Generate all successors of all k states.
- If a goal is found, stop.
- Otherwise keep the best k successors overall.

Example 2.4.1. For the problem in Figure 4, run beam search with $k = 2$.

- Start from S.
- Generate A, B, C, keep A, B.
- Generate successors D, E, F, H from retained nodes.
- Keep F, H.

Expanded nodes are: S, A, B, F, H.

2.4.2 Comparison with Hill-Climbing

With one initial state:

- Hill-climbing keeps only the single best child.
- Beam search keeps the best k children.

With k random initial states:

- Hill-climbing runs k independent climbs.
- Beam search lets the k threads share information by selecting the overall best k successors at each stage.

Thus, beam search is broader and more exploratory than hill-climbing.

2.4.3 Beam search with A*

A beam-style modification of A* keeps only the best k nodes in the fringe.

Advantage: Much lower memory usage.

Disadvantage: Neither complete nor optimal.

This illustrates a common AI trade-off: memory savings in exchange for loss of guarantees.

2.5 Simulated Annealing

Simulated annealing is inspired by metallurgy. When a material is heated and then cooled slowly, its atoms may settle into a low-energy crystalline structure. The algorithm mimics this by allowing occasional bad moves early one, then becoming more conservative over time.

Unlike hill-climbing, which always chooses the best successor, simulated annealing:

- randomly selects a successor m,
- always accepts it if it is better,
- sometimes accepts it even if it is worse.

This ability to accept bad moves is what helps it escape local optima.

2.5.1 Generic procedure

1. Set current node n to the initial state.
2. Randomly choose a successor m.
3. If m is better, accept it.
4. Otherwise accept it with probability p.
5. Repeat until a stopping criterion is met.

2.5.2 Acceptance probability

For minimisation, a common choice is:

$$p = \exp\left[-\frac{v(m) - v(n)}{T}\right]$$

where $v(n)$ is current value, $v(m)$ is successor value, T is the temperature parameter.

If the move is only slightly bad, then p is larger. Thus probability decreases exponentially with the badness of the move.

The parameter T decreases over time according to a schedule, such as:

$$T \leftarrow 0.8T.$$

- High T: bad moves are relatively likely to be accepted.
- Low T: bad moves become unlikely.
- Very low T: behaviour resembles hill-climbing.

So the algorithm begins exploratory and gradually becomes greedy.

2.5.3 Metropolis criterion

Some versions compare p with a random number $p' \in [0, 1]$. If $p > p'$, accept the move, otherwise reject it. This provides a stochastic rule for occasional downhill moves.

2.5.4 Practical view

Simulated annealing is powerful and widely used for large-scale optimisation problems such as VLSI layout, factory scheduling, and other hard combinatorial optimisation tasks.

However, performance depends critically on the annealing schedule, and a "slow enough" schedule may be hard to design.

2.6 Genetic Algorithms

Genetic algorithms are inspired by evolutionary biology. Key mechanisms are selection, crossover, and mutation. They can be viewed as a stochastic form of beam search operating on a population.

Basic terminology

- A state is an **individual**.
- The set of current individuals is the **population**.
- The evaluation score is the **fitness** $f(n)$.
- Better individuals are more likely to reproduce.

Representations

Each individual is encoded as a string. For 8-queens, one encoding is a vector of row positions by column, e.g. $(3, 2, 7, 5, 2, 4, 1, 1)$ means column 1 has a queen at row 3, column 2 has a queen at row 2, and so on.

2.6.1 Fitness function

In the 8-queens example, fitness is the number of **non-attacking pairs of queens**. There are $\binom{8}{2} = 28$ possible pairs, so the maximum possible fitness is 28.

2.6.2 Main operators

- **Selection**: Individuals are selected for reproduction with probability proportional to fitness. Thus, fitter individuals are more likely to contribute genetic material to the next generation.
- **Crossover**: Choose a crossover point and swap suffixes of two parent strings. This creates children that combine parts of both parents.
- **Mutation**: Randomly change bits or entries with small probability. Mutation prevents the population from becoming too homogeneous and introduces fresh variation.

Figure 5: Operators applied in 8-Queens example.

2.6.3 Pseudocode structure

1. Start with population $S_1, \ldots, S_N$.
2. Compute fitness values $f(S_i)$.
3. Define reproduction probabilities:

$$P_i = \frac{f(S_i)}{\sum_{j=1}^{N} f(S_j)}.$$

4. Repeatedly select parents according to P_i.

5. Apply crossover to generate children.
6. Mutate children with small probability.
7. Replace old population with the new one.
8. Repeat until a sufficiently fit individual appears or an iteration limit is reached.

2.6.4 Discussion

Genetic algorithms combine:

- **uphill tendency** through fitness-based selection
- **random exploration** through crossover and mutation
- **parallel search** through the maintenance of a population

Advantages:

- Easy to implement
- Naturally parallel
- Often effective on difficult optimisation tasks

Limitations

- Not complete
- Not optimal
- Highly dependent on the representation and fitness design

3 Summary

3.1 Definitions

- $f(n) = g(n) + h(n)$
- Admissible heuristic: $h(n) \leq h^*(n)$
- Dominant heuristic: $h_2(n) \geq h_1(n)$
- Consistent heuristic: $h(n_i) \leq c(n_i, n_j) + h(n_j)$

3.2 Key Theorems

- h is admissible $\implies$ A* is optimal
- Consistence $\implies$ Admissibility
- h is consistent $\iff$ f is non-decreasing along paths
- h_2 dominates h_2 $\implies$ A* with h_2 expands fewer nodes
- A relaxed problem is solved optimally $\implies$ that solution cost is an admissible heuristic for the original problem
- Simulated annealing finds the global optimum if temperature T is lowered slowly enough

3.3 Standard Comparisons

- UCS is A* with $h(n) = 0$
- BFS is A* when $f(n) = \text{depth}(n)$ with BFS-style tie-breaking
- BFS is UCS when step costs are uniform
- Hill-climbing keeps 1 state; beam search keeps k states; genetic algorithms keep a population.

Table 1: A* Search vs Local Search

Aspect	**A* Search**	**Local Search**
Problem type	Designed for problems where a path matters	Designed for problems where only the final state matters
Solution quality focus	Exact solution cost matters; optimality is important	Finding a good solution efficiently is more important than perfect guarantees
Heuristic requirement	Usually requires a good admissible or consistent heuristic	Does not rely on admissible/-consistent heuristics in the same way
Main strength	Correctness and strong theoretical guarantees	Scalability and low memory usage
Main weakness	Exponential time and especially exponential space	Loss of completeness and optimality guarantees
Best suited for	Problems where guarantees and optimal paths are important	Huge search spaces where exact search is impractical
Core trade-off	Informed systematic search gives guarantees but can be expensive	Local/stochastic search sacrifices guarantees for practicality

Table 2: Comparison of Local Search Methods

Algorithm	**States Kept**	**Main Idea**	**Advantages / Characteristics**	**Guarantees**
Hill-Climbing	One state	Always chooses the best local move	Very memory efficient; simple iterative improvement; easily trapped in local optima	Not complete; not optimal
Beam Search	k states	Keeps the best k candidate states at each stage	Broader than hill-climbing; memory bounded by k; may discard the path to the global optimum	Not complete; not optimal
Simulated Annealing	One state	Allows occasional bad moves to escape local optima	Can escape local optima; performance depends on cooling schedule	Theoretically complete/optimal only under sufficiently slow cooling
Genetic Algorithms	A population	Uses selection, crossover, and mutation for stochastic search	Parallel and stochastic exploration; quality depends strongly on encoding and fitness function	Not complete; not optimal